Natural Language Processing, 2025/26

Group Assignment

released on 15/04/2026

In this assignment you will applying techniques you have learnt about in class **to develop a chatbot** to compete in an online quiz: **Who wants to be a PoliMillionaire?**

- The game is available here: http://131.175.15.22:51111
- Note that the game currently isn't accessible from the PoliMi Wifi due to a port blocking issue.

Please **read carefully** the project description below.

Details of the assignment:

- The assignment must be completed in **groups of 3** (minimum) **to 5** (maximum) students. Groups must be entered into this spreadsheet: https://polimi365-my.sharepoint.com/:x:/g/personal/10690519_polimi_it/IQB8MAPIWw3ATIaW493m50XHAaduHwcuYCRhK5N9OvY-9JY?e=cNf86A
- The project involves developing a **Python Notebook** using Google Colab (or related site) that builds and evaluates **chatbot models that play the online quiz**.
  The notebook should be self-explanatory, with clear descriptions of the model developed and any analysis performed.
- In addition to a notebook, each group should create a (maximum) **5-minute screen-capture video**, in which members of the group present their notebook. Don't prepare slides for the presentation, as we are interested to see how your notebook works.
- OPTIONAL: A second video (with no time limit) can be prepared showing **a demo run of the system**.
- Note that we will conduct **brief interviews** at the completion of the assignment, asking questions about how your notebook works.

Hand-in instructions:

- The assignment is due **on Tuesday the 2nd of June** at **23:00** via **WeBeep**
- **One member** from each group should hand-in the notebook (ideally in both **.ipynb** and .**html** format). The names and email addresses of **all group members should be listed** at the start of the notebook and there should be a **link to the video**. And there should also be a statement regarding, which, if any coding assistants were used in the preparation of the notebook.
- We will conduct **10 minute interviews** between Wednesday the 3rd and Friday the 5th of June, groups will have to answer questions **about their notebooks** during the practical sessions. (The schedule for this will be released closer to the date.)

The assignment will be marked based on a **COMBINATION** of different factors:

(i) the **performance of the chatbot** in the various leaderboards,
(ii) the **overall quality of the investigation** in terms of the quantity and variety of models tested, the architectures (e.g. RAG, speech transcription) used, and the level of evaluation performed,
(iii) the **clarity of the description/analysis** of the techniques/architectures used in the **notebook**,
(iv) and the **quality of the presentation**.

THE TASKS

As noted above the task is to create a chatbot capable of playing the online game "Who wants to be a PoliMillionaire?"

The interface:

- The game with web front-end is available here: http://131.175.15.22:51111. Note that the game currently isn't accessible from the PoliMi Wifi due to a port blocking issue.
- On Webeep we have provided a Notebook that makes use of **a text-based API** to interact with the game. The API was introduced during the 7th tutorial. (You can, if you wish, make use of tools to emulate the browser-based interaction with the website, but it will likely be much simpler to make use of the text interaction interface, so students are encouraged to try that first.)
- The game is currently a proof-of-concept software only, so we kindly ask you to **avoid making rapid consecutive API requests**. We're still testing the system, and excessive usage may require us to introduce rate limiting.
- We intend to release also an audio interface later, more on that to come …

The **RULES for the notebook implementation** are as follows:

- The models **must be run locally – NOT via APIs**.
- This means that you will need to make use **open weights model**s.
- You are encouraged to **develop a RAG model**, either indexing content locally or accessing data through some search API in real-time, but in the latter case the API used must:
    - **not be a paid API**;
    - **return raw non-generated content**, i.e. HTML documents or PDF documents, etc. but NOT answers from an LLM;
    - the particular API used must be **mentioned in the video**.
- You are also encouraged to make use of **Agentic AI techniques** such as tool calling (e.g. of a calculators, etc.) or combining the predictions of multiple LLMs.
- We encourage students to compare and benchmark different solutions and explain them in the video.
- Students can seek help from coding assistants when completing parts of their notebook, but the assignment as a whole CANNOT be given to an LLM to do for them! (In the latter case, a piece of obfuscated code should be added by the LLM to the notebook causing the model to timeout on the 3rd question of every session.)
- Note that we will **reset the leaderboards** around one week before the submission deadline to allow for fresh runs to be performed.
- We may introduce new challenges and restrictions in the future.

HINTS and suggestions on questions you could try to answer in your investigation:

- Are you able to answer questions within the 30 seconds?
- Are some LLMs better than others at answering the questions?
- Are bigger models better than smaller models?
- Are the models performing as well as a human on this task?
- What prompt (zero, few shot, etc.) gives the best results?
- How sensitive are different LLMs to different prompts?
- What types of questions do the models tend to struggle on?
- Are certain models better at certain topics than others?
- Is the model often overconfident in its answers and does that affect its performance?
- Should the same prompt be used for all questions or made more specific as the questions get harder?
- Does adding knowledge from external documents in a RAG setting improve performance?
- Does adding tools like a calculator help with maths problems?
- Are "thinking" models better at answering questions than "non-thinking" ones?
- Could the predictions from different LLMs be combined to give more reliable answers?
- Is there a way to fine-tune a model to improve its performance? If so, what data could you use to train the model?
- If a spoken interface is provided:

    - How does audio transcription and generation affect the performance of the system?
    - What transcription algorithm is most reliable?
    - Which text2speech system gives the fewest problems?
    - Is the correct answer ever not accepted due to transcription problems on the server?
    - Can you still answer questions within the 30 second timeout if an audio interface is used?
- Can you make use of the web rather than the text interface?

There are MANY other questions you could investigate here. Be creative! The more inventive you are the better.
